SDU Bağdar

Smart campus platform for SDU University

Project report: requirements, design, database, testing and user documentation

Course	CSS 217, Assignment 2
Architecture	Microservices behind one gateway (REST / JSON)
Stack	Java 17+ (JDK HTTP server, no framework), Python 3.11+ (standard library), SQLite, HTML / CSS / JavaScript
Design patterns	Strategy, Adapter, Decorator (plus a second Strategy in ProjectForge)
Repository	github.com/iencislennon/Campus_Artery-

Contents

1. Requirements specification
2. Software design
3. Database design
4. Testing documentation
5. User documentation
6. Assignment checklist and gaps that were closed

1. Requirements specification

1.1 Problem statement and goals

Students and teachers at SDU lose time on three everyday problems: finding a free classroom, reaching a teacher during office hours, and getting broken equipment fixed. Information about student initiatives and clubs is scattered over chats. **SDU Bağdar** puts these into one web application, signed in with an SDU e-mail address.

Goals

- Show which rooms of the real SDU layout (blocks D to I) are free right now, filtered by building, floor, kind and capacity.
- Let teachers publish office hours and let students book them manually or automatically, using a scheduling strategy chosen at request time.
- Let any user report a facility problem, detect duplicate reports and let teachers drive them to resolution.
- Let students discover initiatives, follow them, join clubs, answer club surveys and keep a profile with notification settings.
- Keep the system maintainable: layered services, one database per service, three design patterns visible in working code.

1.2 Target users and roles

Role	How the account is created	What the role can do
Student	First sign-in with an `@sdu.edu.kz` address and an e-mailed verification code (device registration).	Browse rooms, teachers, initiatives, clubs; book office hours; report problems and see own reports; follow initiatives, join clubs, answer surveys; edit own profile.
Teacher	"Register as teacher" form; protected by an invite code in production (`TEACHER_INVITE_CODE`).	Everything a student can, plus: manage own office and office hours; review all facility reports and change their status.
Club owner (a user who created a club)	Creating a club with an approved document makes the caller its owner.	Edit the club page, create / edit / delete surveys, read survey results and uploaded files.

1.3 Functional requirements

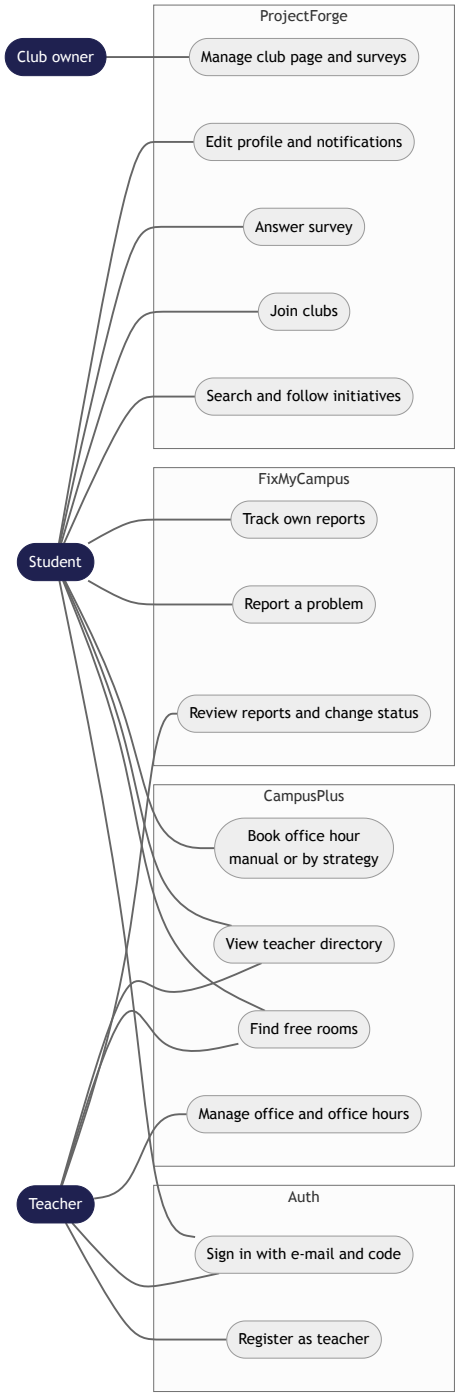
ID	Module	Requirement
FR-1	Auth	Sign in with an SDU e-mail. An unregistered device receives a 6-digit code by e-mail (valid 10 minutes, one request per minute). A registered device signs in without a code.
FR-2	Auth	Register teachers (invite code required unless disabled for development). Role is stored in a signed token and re-checked by every service.
FR-3	CampusPlus	List buildings and rooms; show only rooms free now; filter by building, floor, kind (Classroom / Barrel), minimum capacity, free-text query.
FR-4	CampusPlus	Free-room data comes from the university timetable system through the Adapter.
FR-5	CampusPlus	Teacher directory; teacher registers or updates own office.
FR-6	CampusPlus	Teacher creates office-hour slots; overlapping slots are rejected (409).
FR-7	CampusPlus	Student books a slot manually, or automatically by strategy `first`, `deadline` or `preferred`. One booking per student per slot; capacity is enforced.
FR-8	FixMyCampus	Submit a report (room, category, description). The room is validated by asking CampusPlus.
FR-9	FixMyCampus	Validate, audit, de-duplicate and notify through the Decorator pipeline.

FR-10	FixMyCampus	Teacher moves a report through OPEN, IN_PROGRESS, RESOLVED; the reviewer becomes the assignee. Users see only their own reports and notifications.
FR-11	ProjectForge	Create, search and follow initiatives (Project, Event, Party, Research); recommendations ranked by a runtime-selected strategy.
FR-12	ProjectForge	Create clubs (document checked by the AI validation service), join / leave, club deadlines, club event page.
FR-13	ProjectForge	Club owner builds surveys (short answer, file upload, choice, checkbox); members answer once; owner reads results.
FR-14	ProjectForge	Profile with contacts, photo and four notification switches; feed only shows enabled kinds.
FR-15	All	Every invalid request returns one JSON error shape `{message, code}` with a proper HTTP status.

1.4 Non-functional requirements

ID	Category	Requirement
NFR-1	Security	Passwords hashed with PBKDF2-HMAC-SHA256 (120k iterations, random salt). Verification codes are stored hashed. Tokens are HS256 with expiry; the secret comes from `CAMPUSOS_SECRET` in `.env`, never from code. Roles are read from the token, never from the request body.
NFR-2	Reliability	Every outgoing service call has a timeout. An unreachable service gives `503` with a readable message, never a hang. `/health` reports each service as ok / degraded / down.
NFR-3	Maintainability	Presentation, application, domain and infrastructure layers; no cross-service SQL; each service has an `explanation.md` and a database UML.
NFR-4	Portability	JDK 17+, Python 3.11+, SQLite; no application framework. One command (`scripts/start-all.sh`) builds and starts everything.
NFR-5	Persistence	All data is stored in SQLite files under `data/` and survives restarts.
NFR-6	Usability	Responsive web interface; clear inline error messages; no technical error text shown to users.
NFR-7	Observability	Every request is logged as `[service] METHOD path -> status (ms)`; logs are in `build/logs/`.

1.5 Use cases



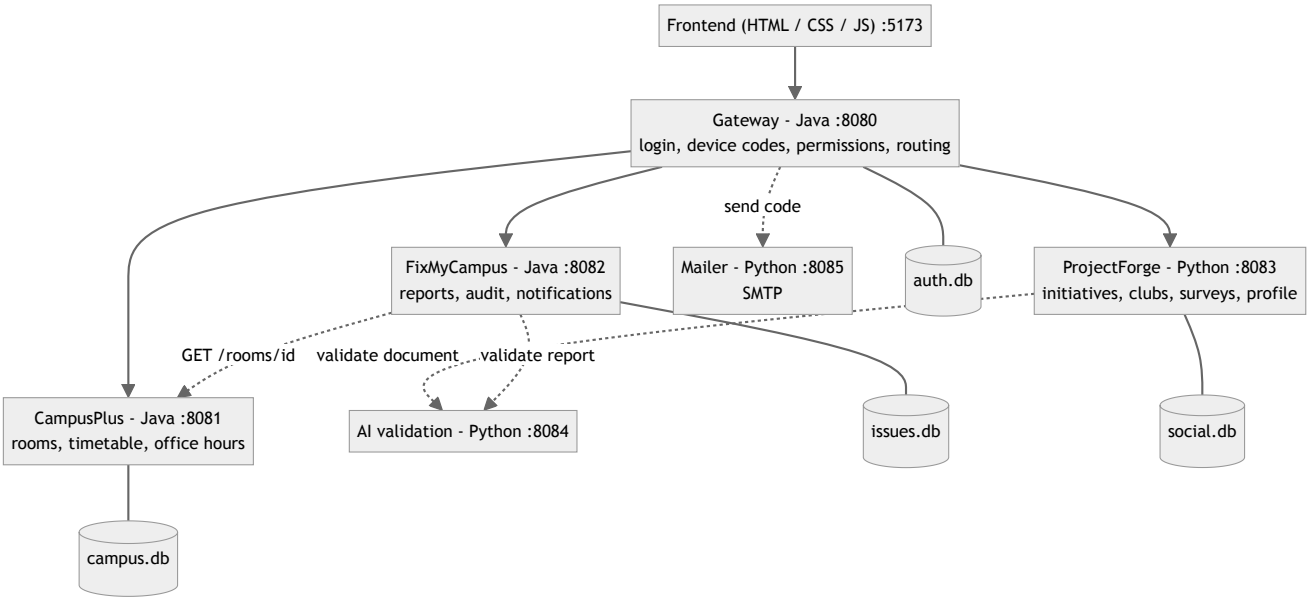
Use case	Actor	Main flow	Alternative / error flows
UC-1 Sign in	Student, Teacher	1. Enter SDU e-mail. 2. Press "Send verification code". 3. Code field appears; enter the 6-digit code from the e-mail. 4. Press "Sign in". The device is remembered.	Not an `@sdu.edu.kz` address: 422. Wrong or expired code: 401. Second code request within a minute: 429. Registered device: no code needed.
UC-2 Find a free room	Student	Open Rooms, choose building / floor / kind / capacity; the list shows rooms free now and until when.	No match: empty list message. Outside 08:30 to 18:00 the 08:30 state is used.
UC-3 Book office hour	Student	Open a teacher, pick a slot, or choose a strategy (first / deadline / preferred) and let the system pick.	Slot full or already booked: 409. Unknown strategy: 422. No slot satisfies the strategy: clear message.
UC-4 Manage office hours	Teacher	Register office, add slots (day, start, end, capacity).	Overlap with an existing slot: 409. Student attempts it: 403.

UC-5 Report a problem	Student, Teacher	Pick room, category, write description; the pipeline validates, links duplicates, saves, notifies.	Room missing: 422. Description too short / long or unknown category: 422 and an audit entry `REJECTED`. CampusPlus down: 503.
UC-6 Review reports	Teacher	Open the review queue (`scope=all`), set status; the reporter is notified.	Student attempts it: 403.
UC-7 Initiatives and clubs	Student	Search, follow, get recommendations with a strategy; join a club; answer its survey.	Missing title or summary: 422. Second survey answer overwrites the first.
UC-8 Run a club	Club owner	Create club with a document, edit event page, create surveys, read answers and uploaded files.	Document rejected by AI validation: 422. Non-owner: 403.

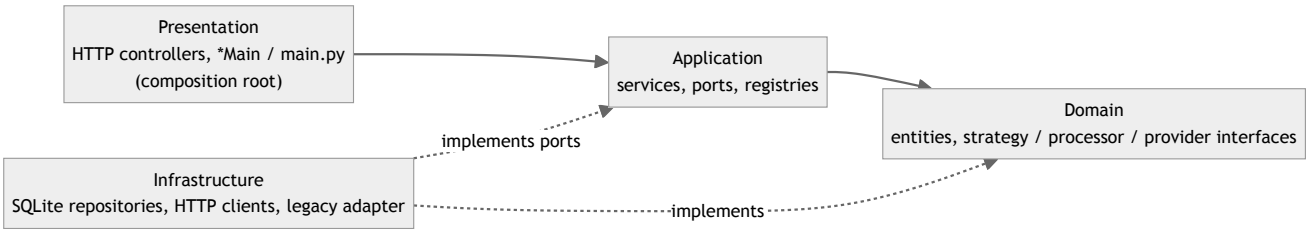
2. Software design

2.1 Architecture

The browser talks only to the **gateway**. The gateway authenticates, checks the role against an access policy and forwards the request to the owning service with the bearer token. Each service owns its database; remote entities are referenced by id and verified over HTTP.



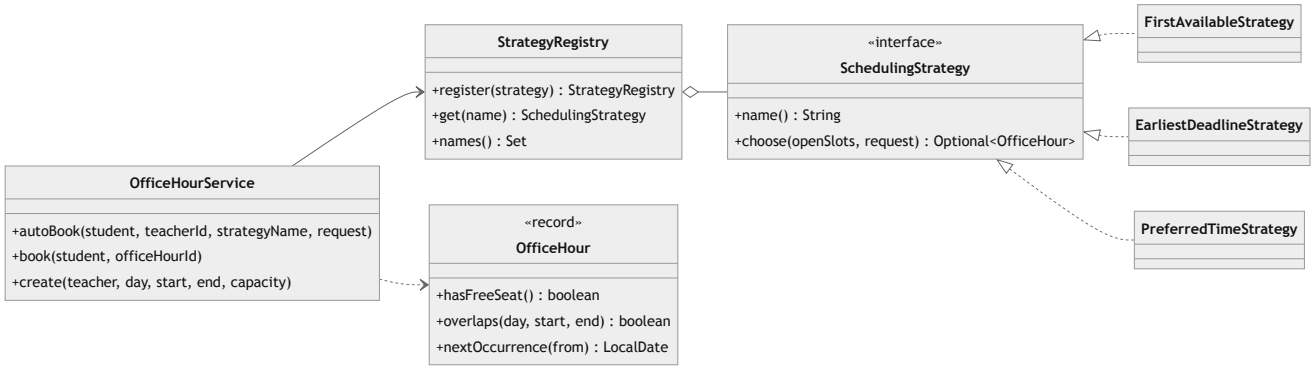
Layers inside every service



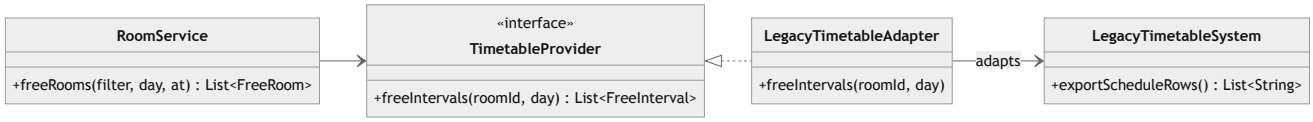
Layer	Responsibility	Example classes
Presentation	Parse HTTP, check token and role, map errors to JSON, wire everything together.	`CampusPlusMain`, `GatewayMain`, `ProxyHandler`, `server.py`
Business logic (application)	Rules and operations: booking, free-room search, report submission, recommendations.	`OfficeHourService`, `RoomService`, `ReportService`, `AuthService`, `services.py`
Domain model	Entities and the interfaces the patterns are built on.	`Room`, `OfficeHour`, `Report`, `SchedulingStrategy`, `ReportProcessor`, `TimetableProvider`
Data access	All SQL lives here, behind repository interfaces.	`SqliteRoomRepository`, `SqliteReportRepository`, `SqliteUserRepository`, Python repositories
Shared plumbing	JSON, tokens, HTTP server, DB access, service client, errors.	`services/common-java`

2.2 UML class diagram

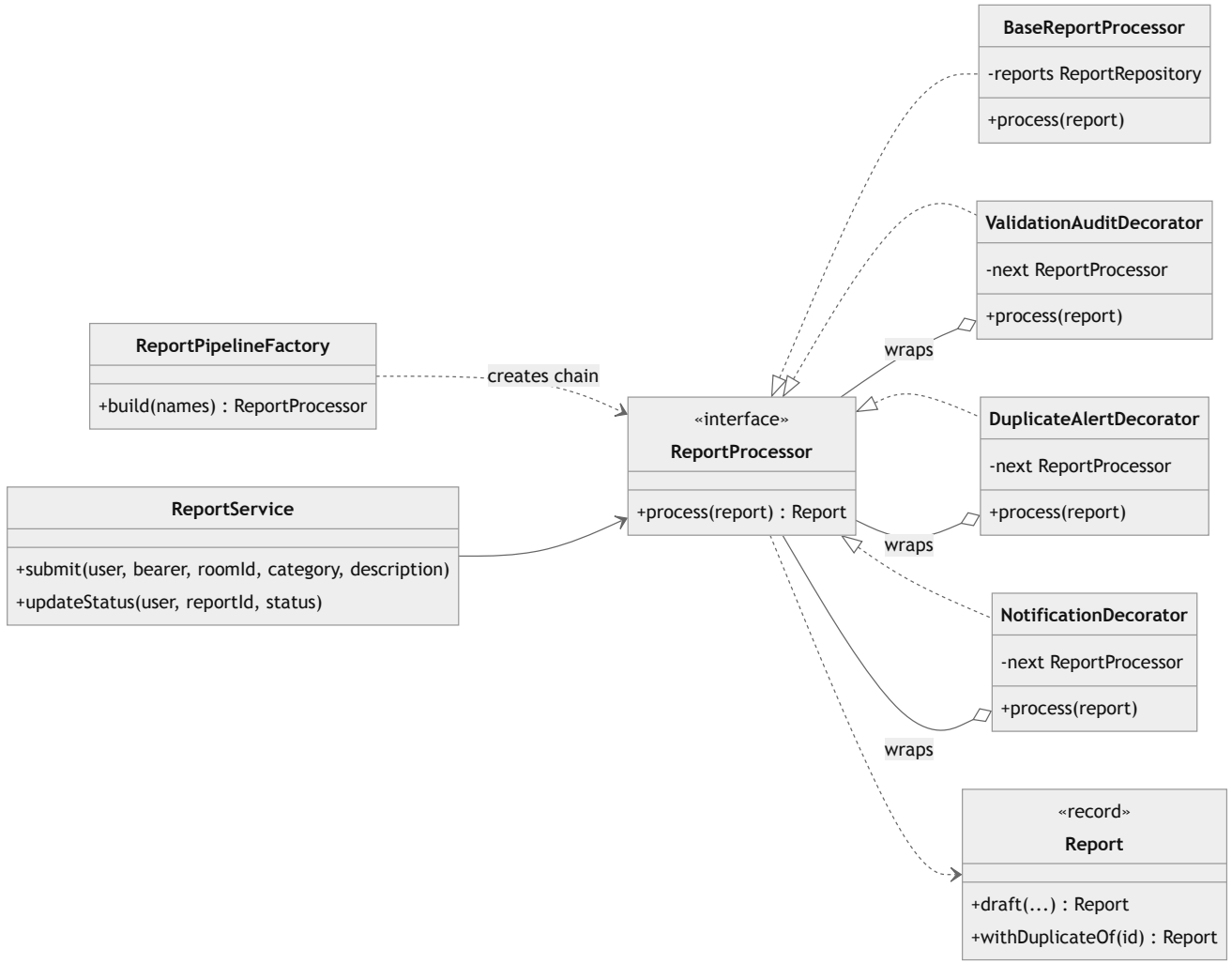
CampusPlus: Strategy



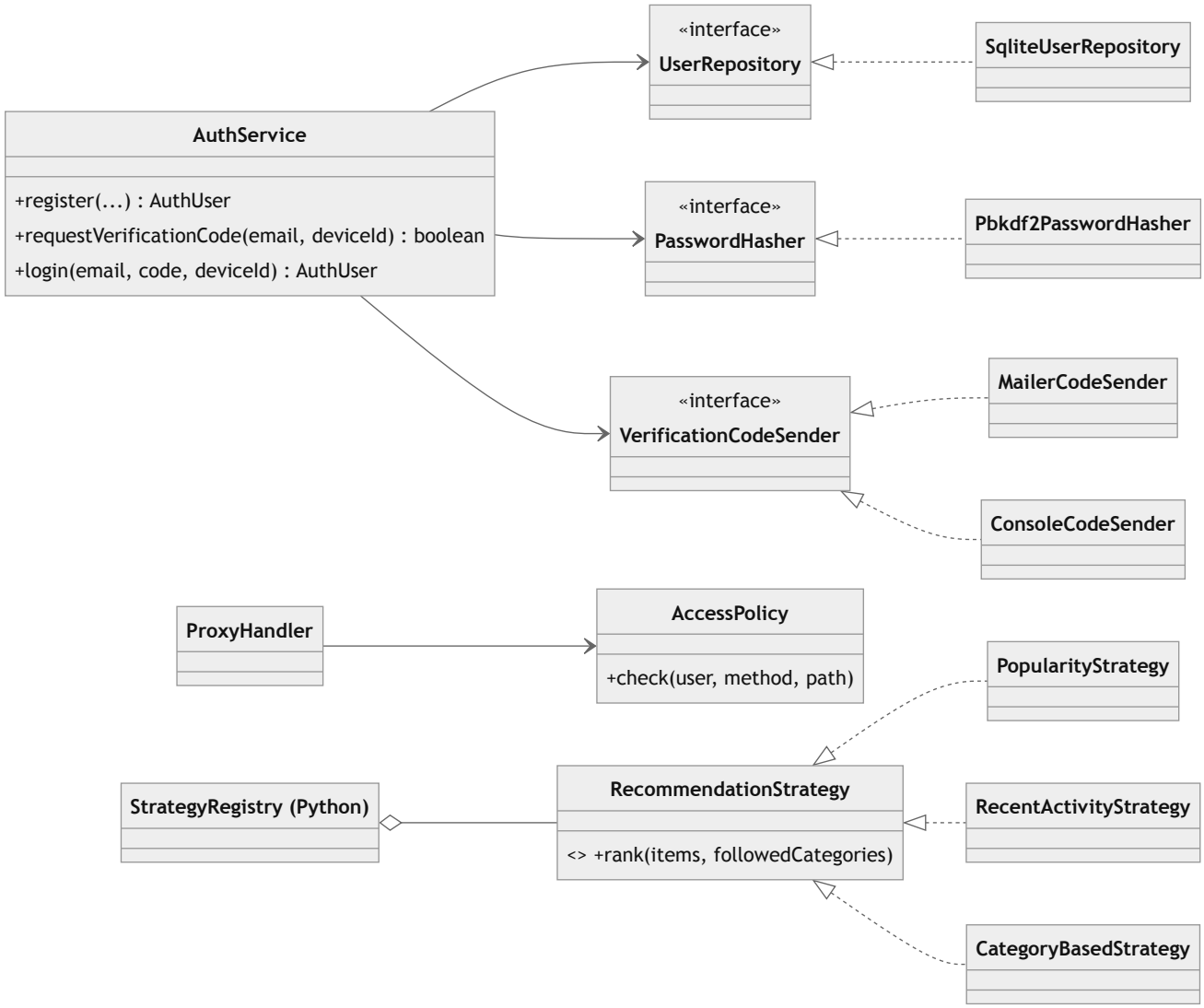
CampusPlus: Adapter



FixMyCampus: Decorator



Gateway and ProjectForge



2.3 How the three required design patterns are used

Strategy: office-hour scheduling (CampusPlus)

Requirement	Where it is met
Common strategy interface	`SchedulingStrategy` with `name()` and `choose(openSlots, request)`.
At least three concrete strategies	`FirstAvailableStrategy` ("first": earliest slot of the weekly order), `EarliestDeadlineStrategy` ("deadline": soonest slot that still happens before the student's deadline), `PreferredTimeStrategy` ("preferred": start time closest to the preferred time).
Select or switch at runtime	The request `POST /office-hours/book` carries `strategy`. `StrategyRegistry.get(name)` resolves it per request; an unknown name returns 422 listing the valid names.
New strategy without rewriting the client	`OfficeHourService` only knows the interface and the registry. A new algorithm is one new class plus one `register(...)` call in `CampusPlusMain`. Unit test "*a new strategy plugs in without changing existing code*" registers a fourth strategy (`last`) and uses it while the other three keep working.

ProjectForge applies the same pattern for recommendations (`popularity`, `recent`, `category`); it is an extra, not needed for the assignment.

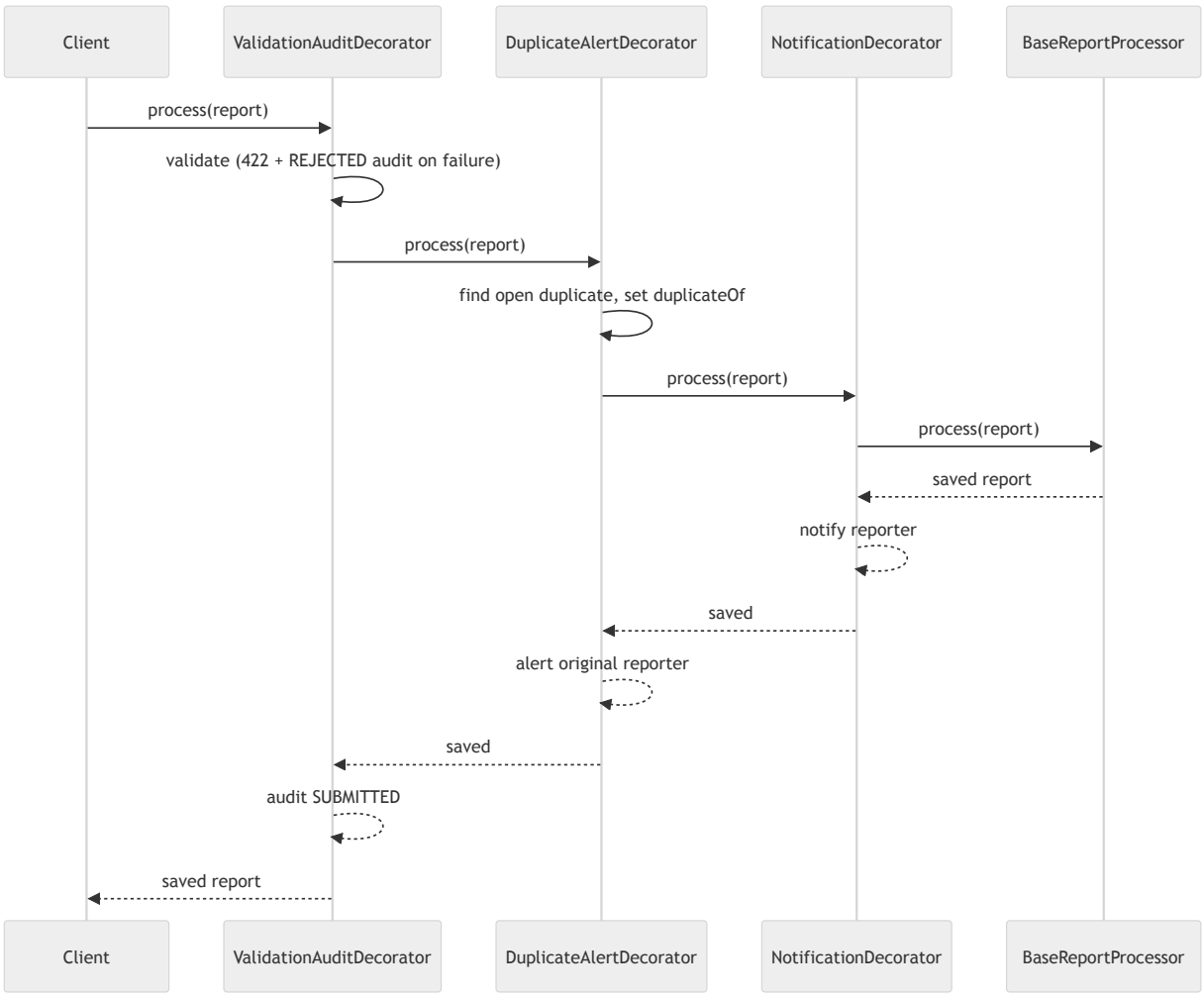
Adapter: SDU timetable import (CampusPlus)

Requirement	Where it is met

Component with an incompatible interface	`LegacyTimetableSystem` simulates the university system. It exports pipe-separated text rows `ROOM|WEEKDAY|HHMM_FROM|HHMM_TO|COURSE` with numeric weekdays and `HHMM` integers.
Adapter makes it compatible	`LegacyTimetableAdapter implements TimetableProvider`. It parses the rows and returns typed `FreeInterval` objects between 08:30 and 18:00.
Original component not modified	`LegacyTimetableSystem` has no dependency on the application and no change was needed to use it.
System uses it through the adapter	`RoomService` depends on `TimetableProvider` only. `GET /rooms` returns `freeUntil` values computed from the legacy rows (verified by the end-to-end test `free_rooms_come_through_the_timetable_adapter`). Replacing the simulation by a real client means writing a new `TimetableProvider`.

Decorator: report processing (FixMyCampus)

Requirement	Where it is met
Common component interface	`ReportProcessor.process(Report)`.
At least three concrete decorators	`ValidationAuditDecorator` (checks category and description length, audits `REJECTED` / `SUBMITTED`), `DuplicateAlertDecorator` (links `duplicateOf`, alerts the first reporter), `NotificationDecorator` (tells the reporter the report was received).
Each wraps another `ReportProcessor`	Every decorator holds a `next` field of the interface type and delegates to it. The innermost component is `BaseReportProcessor`, which only saves.
Combine multiple decorators	`ReportPipelineFactory.build(names)` stacks them in the order given by `FIXMYCAMPUS_PIPELINE` (default `validation,duplicate,notification`, outermost first). Any subset and any order works.
Add behaviour without modifying the original	`BaseReportProcessor` is never edited. The unit test "*a new decorator wraps the chain without modifying it*" adds a tracing decorator on top of the existing chain.



2.4 Important design decisions

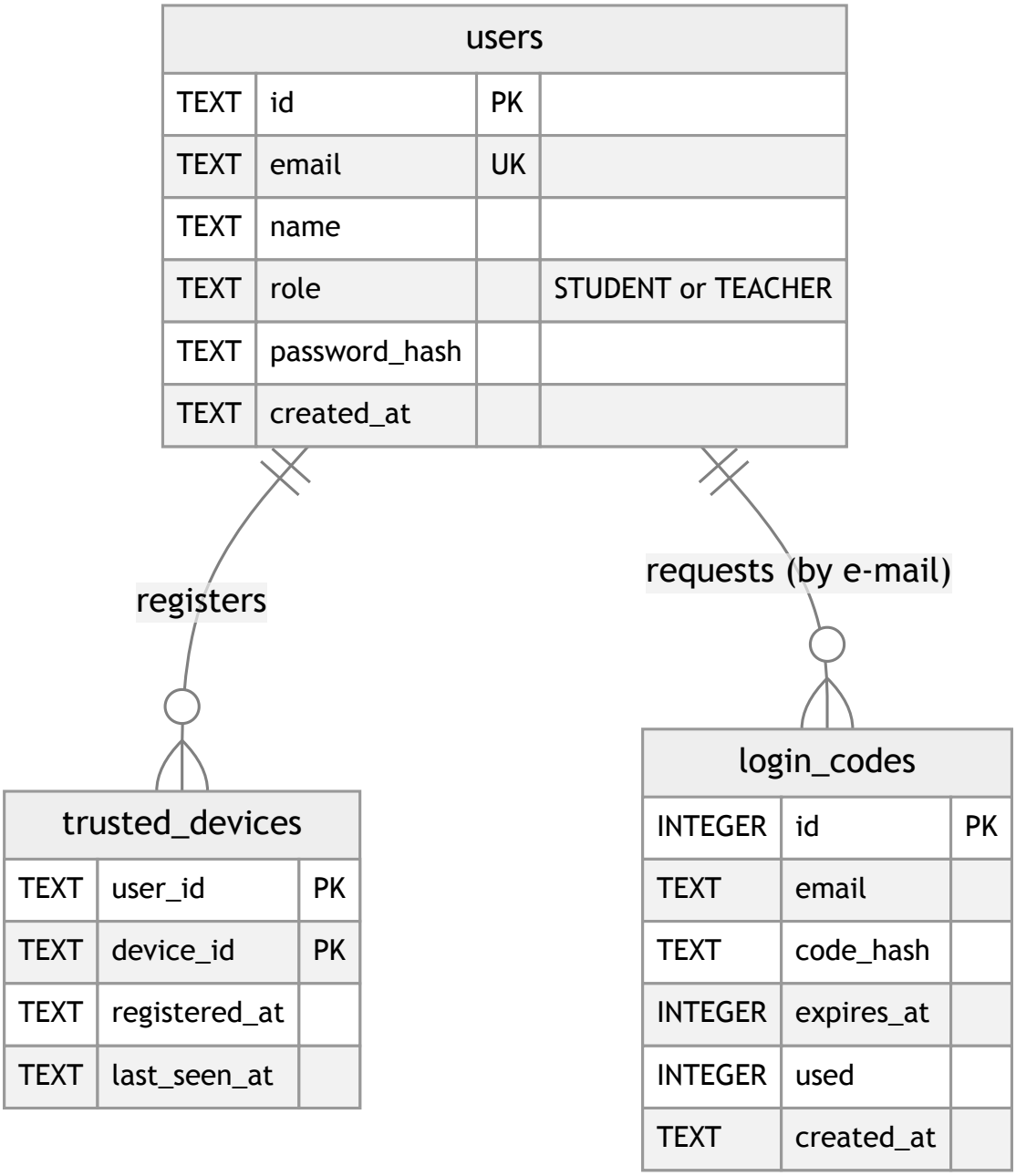
Decision	Reason
Microservices with one gateway	Each module (rooms, reports, initiatives) can change and fail independently; the frontend has a single address.
Database per service, no foreign keys across databases	No hidden coupling. FixMyCampus asks CampusPlus over HTTP whether a room exists instead of reading its tables.
Identity from a signed token, not from the request body	A client cannot grant itself the TEACHER role. Gateway and services both verify.
Device verification instead of passwords for students	Students sign in with an SDU e-mail. A 6-digit code registers the device once; the code is stored hashed, expires in 10 minutes and has a one-minute resend limit.
Ports and adapters (repository, sender, hasher interfaces)	Business rules are testable without a database or mail server; the code sender can be `mailer` (SMTP) or `console` (development).
No application frameworks	JDK `HttpServer` and Python `http.server` keep the dependency list to SQLite and logging jars, and make the layering explicit.
Validation at two levels	Decorators validate content; the service boundary still checks that the room exists, so rules hold even if the pipeline is configured without the validation decorator.
Simulated university data	Timetable, grades and the notification feed are deterministic demo data; the adapter and ports make replacing them a local change.

3. Database design

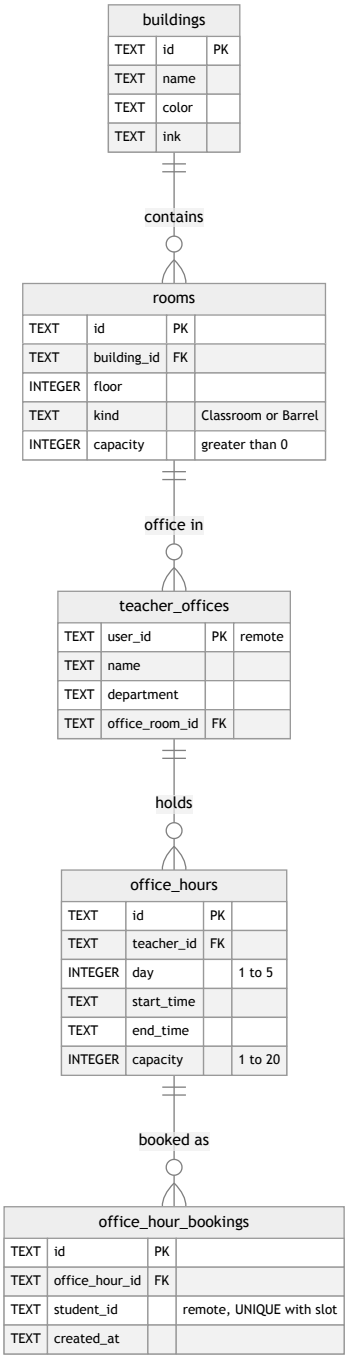
3.1 Data model

Four SQLite files live in `data/`. Columns that point into another service (`user_id`, `reporter_id`, `room_id` in FixMyCampus and so on) are plain text values without foreign keys; they are marked *remote*.

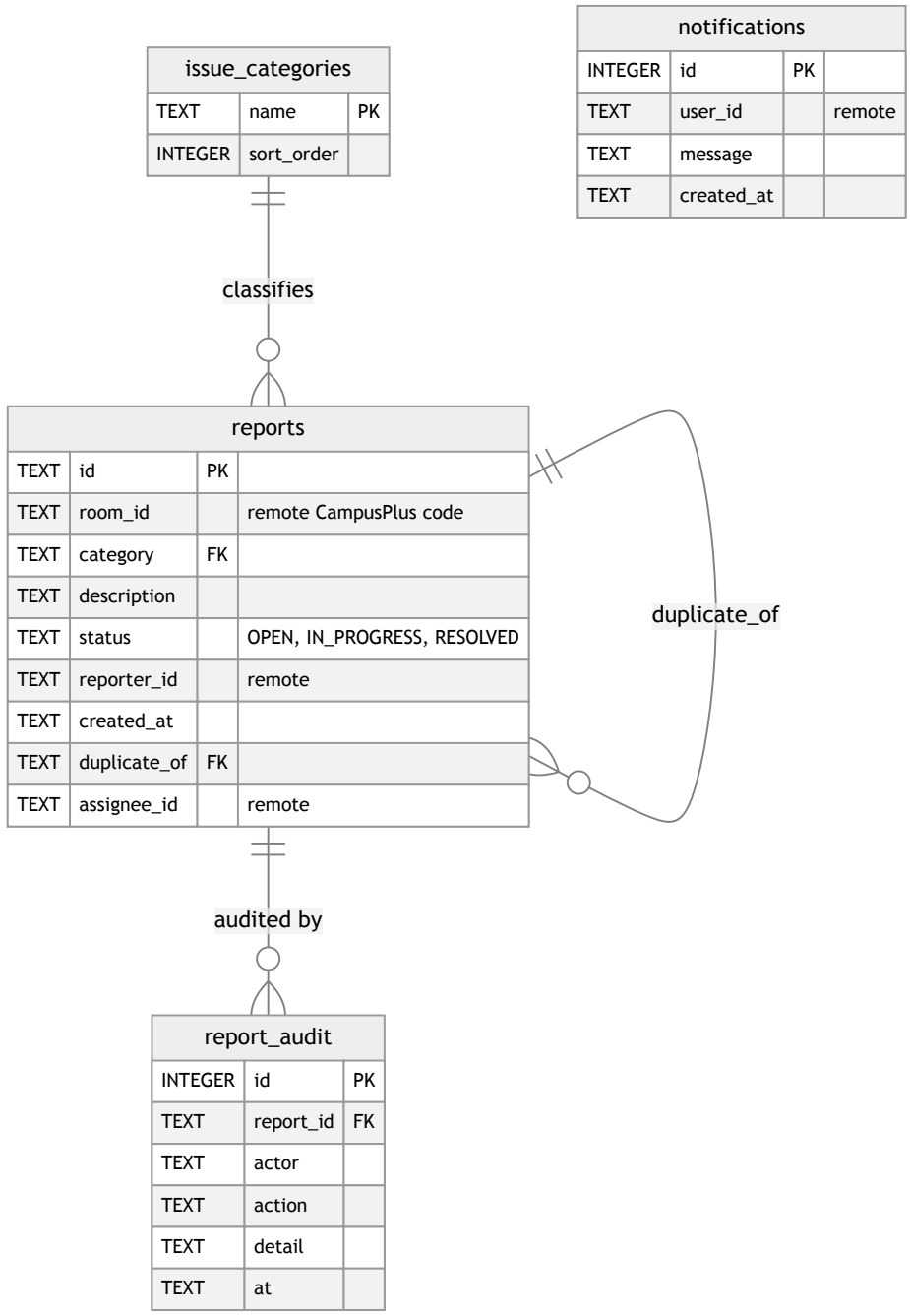
auth.db (gateway)



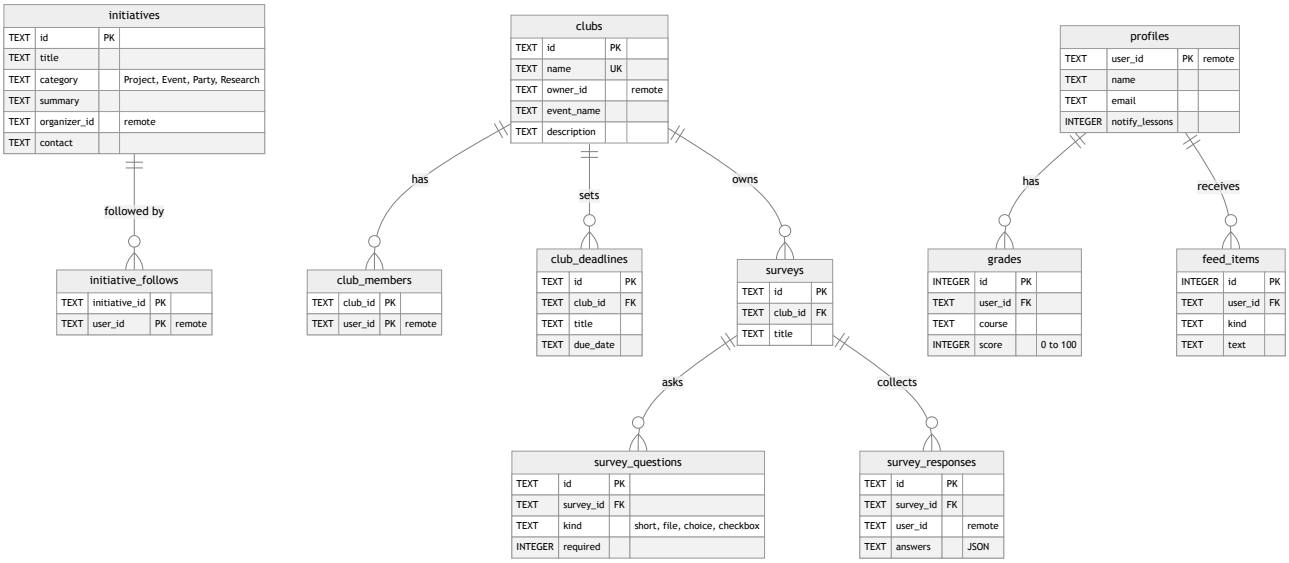
campus.db (CampusPlus)



issues.db (FixMyCampus)



social.db (ProjectForge)



3.2 Main entities and relationships

Entity	Meaning	Relationships
`users`	Student or teacher account.	Referenced by id (inside a token) from all other services. Has trusted devices and login codes.
`buildings` / `rooms`	SDU blocks D to I (colour per block); 90 classrooms plus 11 "barrel" halls.	One building has many rooms. Free-room state is computed from the timetable adapter, not stored.
`teacher_offices` / `office_hours` / `office_hour_bookings`	A teacher's office, weekly slots and student bookings.	1 to N chain. `UNIQUE(office_hour_id, student_id)` prevents double booking; `CHECK` constraints bound day and capacity.
`reports`	A facility problem for a room.	Belongs to one category; may point to an earlier report (`duplicate_of`); has an audit trail.
`report_audit` / `notifications`	History of what happened to reports; messages to users.	`report_id` is nullable so rejected submissions can be audited.
`initiatives` / `initiative_follows`	Student initiatives and who follows them.	Many-to-many with users through the follow table (composite key).
`clubs` and its children	Clubs with members, deadlines and surveys.	Survey to questions to responses; one response per user per survey.
`profiles` / `grades` / `feed_items`	Profile, notification switches and simulated grades / feed.	Created on a user's first profile visit.

3.3 How data is stored and retrieved

- **Storage:** one SQLite file per service in `data/` (`auth.db`, `campus.db`, `issues.db`, `social.db`). Each service creates its schema and, when the database is empty, its demo data at start-up (`CampusSchema`, `FixSchema`, and the Python equivalents). The files are ignored by git, so every installation starts from clean seed data.
- **Access:** only repository classes run SQL, with parameterised statements (user values are never concatenated into SQL). Java services use the `Database` helper: one connection per call, `PRAGMA foreign_keys=ON`, transactions for multi-row changes, and `UNIQUE` violations mapped to HTTP 409.
- **Retrieval:** repositories return domain objects (`Room`, `Report`, ...). Services combine them with business rules; the presentation layer turns the result into JSON.
- **Integrity:** `CHECK` constraints (roles, statuses, capacity, scores), `UNIQUE` keys (e-mail, club name, booking), foreign keys inside a database; cross-service references are validated over HTTP.

- **Secrets in data:** passwords and verification codes are stored only as PBKDF2 hashes.

4. Testing documentation

4.1 Test suites and how to run them

Suite	Location	Tests	Command
Pattern unit tests (Java)	`tests/java/PatternTests.java`	17	`tests/run-java-tests.sh`
End-to-end tests through the gateway	`tests/e2e_test.py`	14	`python3 tests/e2e_test.py -v` (stack must be running)
ProjectForge unit tests	`services/project-forge-python/tests`	2	`python3 -m unittest discover -s services/project-forge-python -t services/project-forge-python`
AI validation unit tests	`services/ai-validation-python/tests`	8	same, with `ai-validation-python`
Mailer unit tests	`services/mailer-python/tests`	4	same, with `mailer-python`

4.2 Test cases for the main functionality

ID	Scenario	Expected result	Test
T-1	All services answer `/health`	Every service reports ok	e2e `test_all_services_healthy`
T-2	Wrong password; calling a route without a token or with a forged token	401	e2e `test_login_rejects_wrong_password_and_protects_routes`
T-3	Teacher registration without the invite code	403 (skipped when the dev stack runs with `SDU_BAGDAR_SKIP_TEACHER_INVITE=true`)	e2e `test_teacher_registration_needs_invite_code`
T-4	Campus contains only blocks D to I	Exactly the six real blocks	e2e `test_campus_has_only_blocks_d_to_i`
T-5	Room filters and barrel halls	Filters by building, floor, kind, capacity	e2e `test_room_filters_and_barrels`
T-6	Office-hour rules: overlap, capacity, double booking, role	409 for conflicts, 403 for students creating slots	e2e `test_office_hours_rules`

T-7	Submit a report, duplicate, notification	201, `duplicateOf` set, notifications created	e2e `test_decorator_pipeline_validates_alerts_and_notifies`
T-8	Invalid report and unknown room	422	e2e `test_validation_and_service_boundary`
T-9	Only teachers review reports	403 for students	e2e `test_only_teachers_review`
T-10	Initiative lifecycle: create, search, follow	Initiative visible and followed	e2e `test_initiative_lifecycle`
T-11	Clubs and profile	Join / leave, profile persisted, feed respects switches	e2e `test_clubs_and_profile`
T-12	Mail contents and recipient rules	Code in the body; non-SDU and header-injection rejected	mailer tests (4)
T-13	AI validation rules	Short, empty or invalid text rejected	ai-validation tests (8)

4.3 Tests for the three design patterns

Pattern	**Test (all in `PatternTests.java` unless noted)**	**What it proves**
Strategy	first picks earliest weekly slot	Concrete strategy 1
	deadline skips slots after the deadline	Concrete strategy 2, including "no slot" case
	preferred picks closest start time	Concrete strategy 3
	preferred without a time yields nothing	Edge case handled safely
	registry switches at runtime and rejects unknown names	Same caller code gives different results depending on the name; 422 for an unknown name
	a new strategy plugs in without changing existing code	Open/closed: a fourth strategy is added from outside
Adapter	legacy rows become free intervals	Incompatible text format is converted; the room is busy during a legacy lesson
	room without lessons is free all day	Adapter returns the full 08:30 to 18:00 interval
	used through the TimetableProvider interface only	Application code needs no knowledge of the legacy class

Decorator	base processor only stores	Undecorated component has no side effects
	validation rejects bad category / description and audits it	Decorator 1 (rejection path stores nothing)
	duplicate links to the open report and alerts its author	Decorator 2
	notification tells the reporter	Decorator 3
	full stack combines all three	Decorators combine and cooperate
	order and subset are configurable	Any order / any subset works; unknown name is rejected
	a new decorator wraps the chain without modifying it	Extension without touching the original component

End-to-end counterparts: `test_free_rooms_come_through_the_timetable_adapter`, `test_strategy_pattern_selects_slots_at_runtime`, `test_decorator_pipeline_validates_alerts_and_notifies`, and `test_recommendation_strategies_switch_at_runtime` (ProjectForge Strategy).

4.4 Invalid input and error handling (live responses)

Input	HTTP	Response
Sign-in e-mail `abc`	422	`{"message":"Enter a valid SDU email address.","code":"VALIDATION_FAILED"}`
E-mail `a@gmail.com`	422	`Use your @sdu.edu.kz email.`
Request without token	401	`Missing bearer token.` (`UNAUTHORIZED`)
Wrong verification code	401	`Email or verification code is incorrect.`
Report with description `x`	422	`AI rejected this report: Describe the issue in at least 10 characters.`
Report for room `Z999`	422	`Room Z999 does not exist.`
Booking with strategy `random`	422	`Unknown scheduling strategy 'random'. Use one of [first, deadline, preferred].`
Student creates an office hour	403	`Your role (STUDENT) cannot do this.`
Initiative with empty title	422	`Title and summary are required.`
Overlapping office hour / double booking	409	Conflict message (e2e T-6)
Second code request within a minute	429	`A code was just sent. Wait a minute before requesting another.`
Mail server not configured	503	`Email delivery is not configured on this server.`
A downstream service is down	503	Readable message; no hang (timeouts)

4.5 Test results

Executed on 22 October 2026 on macOS (JDK 25, Python 3.14) against the running stack.

Suite	Run	Passed	Failed	Skipped
Java pattern unit tests	17	**17**	0	0
End-to-end tests	14	**13**	0	**1 (invite check disabled in the dev `.env`)**
ProjectForge unit tests	2	**2**	0	0
AI validation unit tests	8	**8**	0	0
Mailer unit tests	4	**4**	0	0
Total	**45**	**44**	**0**	**1**

The skipped test (T-3) runs and passes when the stack starts with `SDU_BAGDAR_SKIP_TEACHER_INVITE=false`. Before the test was made configuration-aware it failed with 201 instead of 403 in the development setup; the application behaved as configured, the test assumption was wrong.

```
$ tests/run-java-tests.sh
ok      strategy: first picks earliest weekly slot
ok      strategy: deadline skips slots after the deadline
ok      strategy: preferred picks closest start time
...
ok      decorator: a new decorator wraps the chain without modifying it
17 passed, 0 failed

$ python3 tests/e2e_test.py -v
Ran 14 tests in 0.259s
OK (skipped=1)
```

5. User documentation

5.1 Installation and running

Requirements: JDK 17 or newer (tested on 25), Python 3.11 or newer, `curl` and `openssl`. The needed jars (`sqlite-jdbc`, `slf4j`) are in `lib/`. No internet access or package manager step is required.

```
git clone https://github.com/iencislennon/Campus_Artery-.git
cd Campus_Artery-
scripts/start-all.sh        # builds the Java modules, creates .env with a random secret, starts all 7 processes
open http://localhost:5173  # or open the address in any browser
scripts/stop-all.sh         # stop everything
```

Process	Port	Logs
Frontend (static files)	5173	`build/logs/frontend.log`
Gateway	8080	`build/logs/gateway.log`
CampusPlus / FixMyCampus / ProjectForge	8081 / 8082 / 8083	`build/logs/*.log`
AI validation / Mailer	8084 / 8085	`build/logs/*.log`

5.2 Configuration (`.env`)

Variable	Meaning
`CAMPUSOS_SECRET`	Token signing secret; generated automatically on first start.
`SDU_BAGDAR_SKIP_VERIFICATION`	`false` requires the e-mail code on a new device (default). `true` skips it for development.
`MAIL_DELIVERY`	`mailer` sends real e-mail through SMTP (needs `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`). **`console` prints the code to `build/logs/gateway.log`: use this for a demo or when no mail server is available.**
`SDU_BAGDAR_SKIP_TEACHER_INVITE`, `TEACHER_INVITE_CODE`	Whether teacher registration needs an invite code, and the code itself.
`CAMPUSOS_SEED_DEMO`	`true` creates demo student and teacher accounts (passwords in `.env.example`).
`FIXMYCAMPUS_PIPELINE`	Decorator chain, outermost first. Example: `validation,notification`.

> **Instructor quick path:** in `.env` set `MAIL_DELIVERY=console`, run `scripts/start-all.sh`, sign in with any `@sdu.edu.kz` address, press "Send verification code", then read the code with `grep "\[auth\]" build/logs/gateway.log | tail -1`. Alternatively sign in with the demo accounts `student@sdu.edu.kz` / `teacher@sdu.edu.kz` using `SDU_BAGDAR_SKIP_VERIFICATION=true`.

5.3 Sample data and initial preparation

- **Automatic:** on first start each service creates its tables and fills them (campus layout, teacher offices and office hours, issue categories and sample reports, initiatives, clubs). Delete `data/*.db` and restart to reset everything.
- **Demo accounts** (when `CAMPUSOS_SEED_DEMO=true`): `student@sdu.edu.kz` and `teacher@sdu.edu.kz`.
- **New student:** no preparation; the account is created when the first verification code is requested for a valid `@sdu.edu.kz` address.
- **Teacher:** use "Register as teacher" (invite code `sdu-teacher` unless disabled).

5.4 Main features and how to use them

Feature	How to use it
Sign in	Enter your SDU e-mail. On a new device press **Send verification code**, type the 6-digit code that appears, press **Sign in**. The device is remembered afterwards.
Free rooms	Rooms page: choose block, floor, kind, minimum capacity or search by room number. Each row shows how long the room stays free.
Teachers and office hours	Teachers page: open a teacher and book a slot, or choose a scheduling strategy (*first*, *deadline*, *preferred time*) and let the system choose. Teachers manage their own slots here.
Report a problem	Issues page: pick room and category, describe the problem (10 to 1000 characters). You see duplicates, notifications and the status of your reports. Teachers use *scope: all* to review and change status.
Initiatives	Search, filter by category, follow, create your own; pick a recommendation mode.
Clubs and surveys	Join clubs, see deadlines; owners edit the club page and build surveys (short answer, file, choice, checkbox) and read the answers.
Profile	Contacts, photo and four notification switches (lessons, deadlines, grades, attendance).

5.5 Troubleshooting

Symptom	Cause and fix
Browser: "localhost refused to connect"	The frontend is not running. Run `scripts/start-all.sh`.
"Email delivery is not configured on this server."	SMTP is empty. Set `MAIL_DELIVERY=console` or fill the `SMTP_*` variables and restart.
"A code was just sent. Wait a minute..."	The resend limit is one per minute.
Page still shows an old sign-in screen	Browser cache: hard refresh (Cmd/Ctrl + Shift + R).
Port already in use	Run `scripts/stop-all.sh`, or change the `*_PORT` values in `.env`.

6. Assignment checklist and gaps that were closed

Requirement	Status	Evidence
Strategy: interface, 3+ strategies, runtime switch, extensible	**Done**	Section 2.3; `SchedulingStrategy`; 6 unit tests
Adapter: incompatible component, adapter, original untouched, demonstrated	**Done**	Section 2.3; `LegacyTimetableAdapter`; 3 unit tests + e2e
Decorator: interface, 3+ decorators, wrapping, combinable, no change to original	**Done**	Section 2.3; `ReportPipelineFactory`; 7 unit tests + e2e
Presentation layer	**Done**	Web frontend in `frontend/`, HTTP controllers
Business logic layer	**Done**	`application/` packages
Data access layer	**Done**	Repository interfaces and SQLite implementations
Domain model	**Done**	`domain/` packages
Error handling and validation	**Done**	Section 4.4
Persistence (not in-memory only)	**Done**	SQLite files in `data/`
Separation of concerns	**Done**	Section 2.1 and 2.4
A. Requirements specification	**Done (completed)**	Section 1. **Added:** consolidated FR / NFR lists, roles and a use-case diagram with use-case descriptions (previously only in scattered notes).
B. Software design	**Done (completed)**	Section 2. **Added:** UML for gateway and ProjectForge, layer diagram and a Decorator sequence diagram.
C. Database design	**Done (completed)**	Section 3. **Added:** ER diagrams for all four databases (including `trusted_devices` and `login_codes`, which were missing from the gateway UML) and the storage / retrieval description.
D. Testing documentation	**Done (completed)**	Section 4. **Added:** a Java unit-test suite for the three patterns (there were none), configuration-aware e2e test, live error examples and recorded results.
E. User documentation	**Done (completed)**	Section 5. **Added:** verification-code flow, `.env` reference, sample-data notes and troubleshooting.
At least five meaningful tests besides pattern tests	**Done**	11 e2e tests plus 14 service unit tests
README and diagrams	**Done**	`README.md`, `docs/architecture.md`, `uml-diagram.md`, per-service `docs/db-uml.md`, this report
Runs in the instructor's environment from the instructions	**Done, with a caveat**	One command starts everything. Real e-mail needs SMTP credentials; for a demo set `MAIL_DELIVERY=console` (section 5.2).

Known limitations

- The university timetable, grades and the notification feed are simulated data (the Adapter and the ports isolate this).
- The AI validation service is a local rule-based placeholder; an external provider can be set with `AI_PROVIDER_URL`.
- Verification e-mail needs SMTP credentials; without them use the `console` delivery mode.
- The Java auth code (`AuthService`) has no dedicated unit tests; it is covered by the end-to-end tests and manual checks.